



ANKARA YILDIRIM BEYAZIT UNIVERSITY
GRADUATE SCHOOL OF NATURAL AND APPLIED
SCIENCES
COMPUTER ENGINEERING

DataGen: Matrix Dataset Generation

By

Ali Emre Pamuk 21050111021

Faruk Kaplan 21050111026

Mert Altekın 21050111065

Supervised by

Fahreddin Şükrü Torun

Date

12.10.2024

Abstract

This project addresses the challenges of large-scale matrix generation and optimization by developing a framework for efficient processing, visualization, and analysis. Matrices were collected from various sources, expanded, and optimized using Python-based algorithms. Key properties such as symmetry, sparsity, and statistical measures were extracted and visualized via an interactive GUI developed with Tkinter. A property-based loss function guided the optimization process to improve alignment between generated and original matrices.

The framework demonstrated scalability, resource efficiency, and successful integration of visualization tools like heatmaps and graphs. Challenges such as computational constraints and numerical errors were mitigated through systematic approaches. Future work will focus on integrating machine learning, refining the optimization process, and enhancing the GUI for improved responsiveness and advanced features, paving the way for broader applications in matrix analysis.

Contents

Introduction	3
Methodology	4
Research and Exploration	4
Data Collection	4
Matrix Generation	4
Optimization	5
Graphical User Interface (GUI) Development	6
Challenges and Limitations	7
Matrix Collection and Preparation	7
Feature Engineering and Property Calculation	7
Matrix Generation and Optimization	8
Computational and Resource Constraints	8
Graphical User Interface (GUI) Development	8
Application-Specific Issues	8
Results	9
Graphical User Interface (GUI)	9
Matrix Collection and Generation	9
Feature Extraction and Visualization	9
Optimization and Loss Reduction	9
Resource Efficiency	9
Future Work	10
Machine Learning Integration	10
Graphical User Interface (GUI) Enhancements	10
Feature Selection and Analysis	10
Code Quality and Modularity	10
Numerical Property Calculation	10
Optimization Improvements	10
References	11
Appendix	12

Introduction

The efficient generation and processing of matrices are pivotal in numerous scientific and engineering domains, including data analysis, machine learning, computational simulations, and scientific computing. Matrices serve as fundamental building blocks in these fields, enabling the representation of complex systems and the solving of large-scale computational problems. However, as the scale and complexity of these matrices continue to grow, traditional methods of matrix processing face significant limitations. Challenges such as computational inefficiency, difficulty in preserving critical structural properties, and scalability issues have highlighted the need for more advanced and adaptable solutions.

This research project seeks to address these limitations by developing a robust framework for the generation, manipulation, and optimization of matrices. The framework focuses on creating matrices with varying sizes and complexities while ensuring that essential structural properties, such as sparsity, symmetry, and numerical stability, are preserved. By combining theoretical exploration with practical implementation, the project aims to push the boundaries of computational efficiency and accuracy in matrix processing. The ultimate goal is to provide methods that are both versatile and scalable, capable of addressing the needs of modern computational systems.

To achieve these objectives, the research systematically examines the existing body of knowledge, including algorithms, tools, and techniques for matrix processing. Key references in this domain include foundational works such as:

- **R-MAT Graph Generator:** D. Chakrabarti, Y. Zhan, and C. Faloutsos, R-MAT: A Recursive Model for Graph Mining [2].
- **SPARSKIT:** Youcef Saad’s toolkit for sparse matrix computations, which provides practical resources for handling sparse systems [6].
- **Finite Element Matrix Generation on a GPU:** A. Dziekonski, P. Sypek, A. Lamecki, and M. Mrozowski’s exploration of GPU-based matrix generation [3].
- **J-Orthogonal Matrices:** Nicholas J. Higham’s study on properties and generation methods of orthogonal matrices [4].

These works provided critical insights into the state-of-the-art techniques for matrix generation and manipulation, informing the research methodology and identifying areas for innovation.

The project also involves the collection and exploration of diverse datasets, sourced from publicly available repositories and applications in fields such as multiphysics simulations. Furthermore, the research includes the design and implementation of methods for matrix manipulation and optimization, leveraging both traditional numerical techniques and emerging artificial intelligence (AI)-based approaches.

Methodology

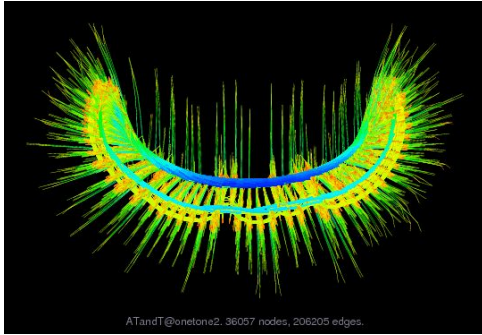
This project utilized a systematic methodology for matrix generation, evaluation, and optimization, designed to address the computational challenges associated with large-scale matrices. The methodology was divided into three main stages: data collection, matrix generation, and optimization. Below, we describe each phase in detail.

Research and Exploration

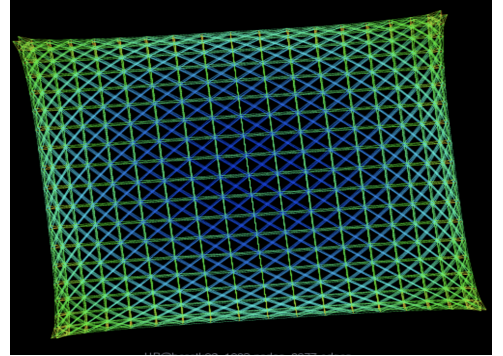
We began by collecting matrices from various sources, including public repositories (e.g., SuiteSparse Matrix Collection) and specialized applications such as FEATools Multiphysics for computational simulations. These matrices were selected to represent diverse structural and numerical properties, including sparsity, symmetry, and non-zero value distributions. The collected matrices served as the foundation for the subsequent stages of the project.

Data Collection

Matrices were collected from various repositories and applications, including open-source platforms such as SuiteSparse Matrix Collection and computational simulation tools like FEATools Multiphysics. These datasets were carefully selected to ensure diversity in structure, sparsity, and numerical properties, thereby providing a comprehensive basis for the research.



(a) onetone2 [5]



(b) bcsstk09 [1]

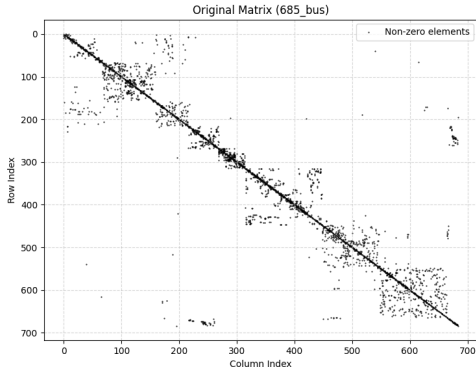
Figure 1: Collected Example Matrices

Matrix Generation

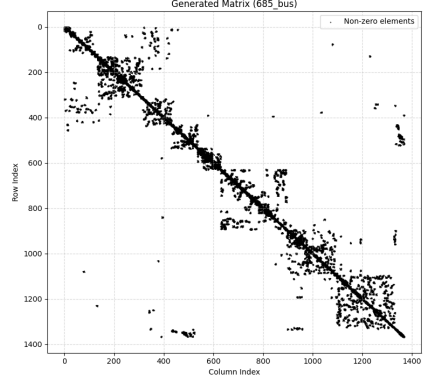
To generate new matrices, we implemented scaling and expansion algorithms. Given an initial matrix $A \in R^{m \times n}$, we used scaling transformations to create matrices of desired dimensions while preserving the essential properties of the original matrix. The expansion process added controlled noise and density adjustments to simulate real-world scenarios. The algorithm for matrix expansion can be expressed as:

$$X_j = \text{expand_matrix}(A), \quad \forall j \in \{1, \dots, k\},$$

Where X_j represents j -th generated matrix, and ‘expand_matrix’ applies proportional scaling and random perturbations to the non-zero entries of A (Algorithm 1)



(a) Original Matrix (685_bus)



(b) Generated Matrix (685_bus)

Figure 2: Test on Example Matrix

Optimization

The optimization phase aimed to minimize the difference between the original matrix A and the generated matrices X_j in terms of specific numerical properties. We defined a property function $P(A)$ that computes a vector of numerical properties for a given matrix A .

$$P(A) = \{p_1(A), p_2(A), \dots, p_r(A)\}$$

where each property $p_i(A)$ represents a numerical measure, such as:

- **Non-zero statistics:** $\min(\text{nnz}(A_i; :)), \max(\text{nnz}(A_i; :)), \text{mean}, \text{std}$,
- **Symmetry checks:** Pattern symmetry and numerical symmetry,
- **Matrix norms:** $\|A\|_1, \|A\|_\infty, \|A\|_F$,
- **Condition number:** $\text{cond}(A, 1)$.

To evaluate the quality of generated matrices X_j , we applied *min-max scaling* to normalize the properties:

$$p_i^{\min} = \min_j p_i(M_j), \quad p_i^{\max} = \max_j p_i(M_j)$$

resulting in a scaling dictionary $\mathcal{S}$ with bounds for each property p_i .

A property-based loss function was used to quantify the difference between A and X_j :

$$\text{Loss}(A, X_j) = \sum_{i=1}^r w_i \cdot \Delta_i(A, X_j)$$

where $\Delta_i(A, X_j)$ measures the scaled absolute difference of property p_i between A and X_j , defined as:

$$\Delta_i(A, X_j) = \begin{cases} \frac{|p_i(X_j) - p_i(A)|}{p_i^{\max} - p_i^{\min} + \epsilon}, & \text{if } p_i^{\max} \neq p_i^{\min}, \\ 0, & \text{otherwise.} \end{cases}$$

Here, w_i is a weight assigned to each property, and $\epsilon > 0$ prevents division by zero.

Finally, we minimized the total loss across all generated matrices using *local search optimization*:

$$\text{TotalLoss}(A, X_1, \dots, X_k) = \sum_{j=1}^k \text{Loss}(A, X_j)$$

The optimization process iteratively perturbed the matrices X_j to improve their alignment with A . Random perturbations were applied to the non-zero entries, and changes were accepted if they reduced the total loss. This process is summarized as:

1. **Initialization:** Start with matrices $\{X_1, \dots, X_k\}$ from the generation phase.
2. **Iteration:**
 - Select X_j randomly.
 - Apply a perturbation: $\tilde{X}_j = \text{Perturb}(X_j)$.
 - Compute the new total loss: L_{new} .
 - Accept $\tilde{X}_j$ if $L_{\text{new}} < L_{\text{old}}$, otherwise revert.
3. **Output:** Optimized matrices $\{\tilde{X}_1, \dots, \tilde{X}_k\}$.

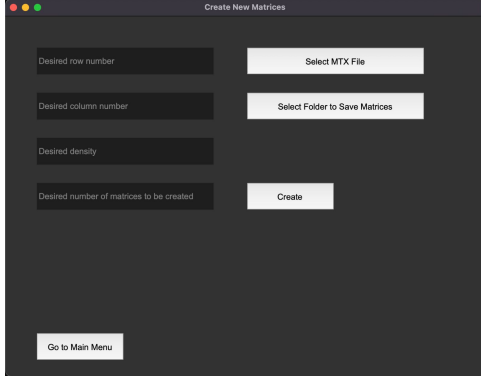
This iterative process ensured gradual exploration of the solution space while maintaining computational efficiency. (Algorithm 2)

Graphical User Interface (GUI) Development

To streamline and simplify the workflow described in the previous sections, we developed a user-friendly Graphical User Interface (GUI). The GUI serves as a comprehensive tool for executing matrix generation, evaluation, and optimization tasks seamlessly. Key features of the GUI include:

- **Matrix Import and Visualization:** Users can import matrices and visualize their structure, including sparsity patterns and numerical properties.
- **Interactive Matrix Generation:** The interface allows users to specify basic parameters for matrix generation.
- **Property Analysis and Comparison:** The GUI computes and displays numerical properties of matrices, enabling easy comparison between original and generated matrices.
- **Export and Integration:** Generated and optimized matrices can be exported for use in external applications or simulations, ensuring compatibility with existing workflows.

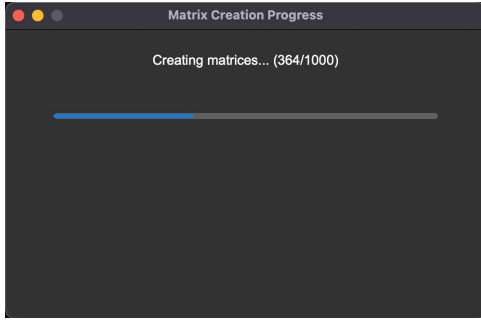
The GUI was built using Tkinter/Python to ensure user-friendly experience. This tool enhances accessibility, enabling both researchers and practitioners to efficiently apply the developed methodology without delving into the underlying codebase.



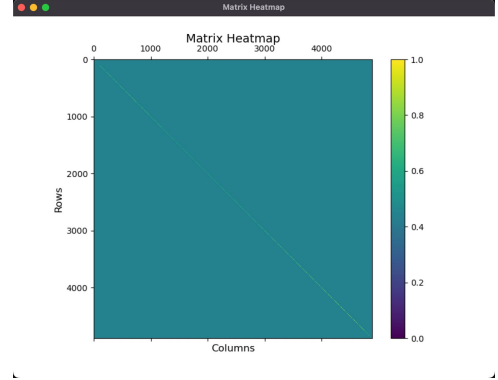
(a) Creation Page of GUI

Property	Value
Number of Rows	130
Number of Columns	130
Number of Nonzeros	1037
Density (%)	6.135094674556213
Pattern Symmetry	False
Numerical Symmetry	False
Minimum Nonzeros per Row	1
Maximum Nonzeros per Row	39
Average Nonzeros per Row	7.976923076923077
Standard Deviation of Nonzeros per Row	7.427045571252195
Minimum Nonzeros per Column	1
Maximum Nonzeros per Column	124
Average Nonzeros per Column	7.976923076923077
Standard Deviation of Nonzeros per Column	18.050981385899295
Minimum Value	-105155.625
Maximum Value	10.52057833807373
Average Value	-4549.538152391431
Standard Deviation of Values	14480.583638122516
Average Distance to Diagonal	39.24530163334428
Number of Diagonals with Nonzeros	119
Bandwidth	125
Number of Structurally Unsymmetric Elements	918
1-Norm	105156.64900381863
Infinity Norm	1084597.375
Frobenius Norm	48878.4552739988
Estimated Condition Number	10798708075.45694

(b) Features Menu



(c) Progress Bar



(d) Heat Map

Figure 3: Glance to GUI

Challenges and Limitations

Throughout the course of this project, we encountered several challenges and limitations, ranging from technical implementation to computational constraints. These issues were addressed systematically, but they highlight areas where further refinement or resources could improve the framework. Below, we categorize and detail the challenges:

Matrix Collection and Preparation

- **Collecting matrices from various sources:** Locating diverse and high-quality matrices with specific properties was time-intensive and required extensive exploration of repositories and applications such as FEATools Multiphysics.
- **Converting data formats to matrices:** Many datasets were stored in non-matrix formats, necessitating preprocessing and conversion to extract usable matrices.

Feature Engineering and Property Calculation

- **Finding matrix features:** Identifying key numerical and structural properties for analysis required significant effort in feature engineering.
- **Mathematically and numerically calculating property values:** Developing robust algorithms to compute properties such as symmetry, sparsity, and norms

was challenging, particularly for large-scale matrices.

- **Encoding categorical features:** Converting categorical properties into numerical values for use in property-based loss calculations was complex and required careful encoding strategies.

Matrix Generation and Optimization

- **Generating expanded matrices:** Scaling matrices, shifting their values, and adjusting their density while preserving key structural properties proved to be computationally demanding.
- **Finding an appropriate loss function:** Designing a property-based loss function that effectively measured the difference between the original and generated matrices required extensive experimentation.
- **Decreasing loss and improving matrix similarity:** Iteratively minimizing the property-based loss to align the generated matrices with the original matrix presented challenges in terms of computational efficiency and convergence.
- **Handling numeric errors:** Issues such as NaN values, infinite values, and memory overflows frequently arose during calculations, requiring additional error-handling mechanisms.

Computational and Resource Constraints

- **Intensive memory and computational resource needs:** The large-scale nature of the matrices required significant computational power and memory, which constrained some aspects of the project.
- **Effective resource usage:** Efficiently managing system resources, such as memory, processing power, and Tkinter objects for the GUI, was a persistent challenge.

Graphical User Interface (GUI) Development

- **Connecting frontend inputs to backend operations:** Integrating user inputs and file uploads with backend processing required careful design and debugging.
- **Adding graphs and heatmaps:** Visualizing matrix properties and patterns in the GUI using heatmaps and graphs demanded both computational resources and responsive design techniques.
- **Effective responsive design:** Ensuring that the GUI was user-friendly, responsive, and functional across different screen sizes and resolutions was a key challenge.

Application-Specific Issues

- **Installation and usage of simulation applications:** Tools such as ANSYS required substantial setup and learning, adding to the complexity of data collection and preprocessing.

- **Limited resources for similar implementations:** The lack of existing implementations and resources specific to this type of matrix generation and optimization project required innovative problem-solving and original methods.

Results

The results of this research project demonstrate the successful implementation and execution of the proposed methodology, achieving the primary objectives. Key outcomes are summarized as follows:

Graphical User Interface (GUI)

- A fully functional and user-friendly GUI was developed using Tkinter, enabling seamless interaction between the user and the matrix processing framework.
- The transfer of inputs and outputs between the frontend interface and backend operations was successfully implemented, ensuring smooth communication and real-time updates.

Matrix Collection and Generation

- Multiple matrices were sourced from online repositories and simulation applications, enriching the dataset with diverse properties.
- Python-based algorithms were employed to generate new matrices by scaling, adjusting density, and introducing perturbations. These generated matrices effectively maintained key properties of the original matrices.

Feature Extraction and Visualization

- Essential matrix features, such as symmetry, sparsity, and row/column statistics, were accurately extracted and calculated for both original and generated matrices.
- Visual representations, including plots and heatmaps, were integrated into the GUI to provide an intuitive understanding of matrix properties and patterns.

Optimization and Loss Reduction

- The property-based optimization algorithm successfully minimized the difference between generated matrices and the original matrices.
- With a relatively low number of iterations, the optimization process achieved a 5% reduction in the property-based loss, demonstrating the effectiveness of the algorithm in improving matrix similarity.

Resource Efficiency

- Efficient resource management was implemented, both in the frontend (e.g., window management for GUI components) and the backend (e.g., computational resource optimization during matrix processing).

Future Work

While this project successfully achieved its primary objectives, there are several areas for further development and improvement. The planned future work aims to expand the functionality of the framework, enhance its performance, and explore advanced applications. The key directions are as follows:

Machine Learning Integration

- Transitioning to the machine learning phase, we plan to train models using the matrices generated during this project. This will enable predictive and automated insights into matrix generation, optimization, and property analysis.

Graphical User Interface (GUI) Enhancements

- The GUI will be improved to incorporate advanced features such as responsive design for various screen resolutions and selective resolution settings, ensuring a better user experience.
- Enhanced visualization tools, including more detailed and interactive graphs and heatmaps, will be integrated to provide deeper insights into matrix properties and patterns.

Feature Selection and Analysis

- Feature selection will be implemented to identify and prioritize the most significant properties. Features with lower importance will be assigned smaller weights or excluded to improve computational efficiency and focus on critical aspects.

Code Quality and Modularity

- The codebase will be refined to enhance modularity, memory efficiency, and execution speed. This includes restructuring key components and optimizing algorithms for scalability.

Numerical Property Calculation

- The calculation of numerical properties will be improved to ensure greater accuracy and robustness. This involves refining algorithms to handle edge cases and minimizing potential numerical errors.

Optimization Improvements

- The cost function used for optimization will be improved to better reflect the desired characteristics of generated matrices. This will include redefining loss metrics and incorporating more nuanced property comparisons.
- More deterministic optimization techniques will be implemented to reduce the reliance on random and stochastic processes. These techniques will aim for greater consistency and reliability in achieving optimal solutions.

References

- [1] Harry Bouwkamp. Bcsstk09 matrix dataset, 1982.
- [2] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. R-mat: A recursive model for graph mining. *SIAM International Conference on Data Mining*, 2004. Accessed from foundational works on graph mining.
- [3] Andrzej Dziekonski, Piotr Sypek, Andrzej Lamecki, and Mirosław Mrozowski. Finite element matrix generation on a gpu. *Computer Methods in Applied Mechanics and Engineering*, 254:224–232, 2013. Exploration of GPU-based matrix generation.
- [4] Nicholas J. Higham. J-orthogonal matrices: Properties and generation methods. *SIAM Journal on Matrix Analysis and Applications*, 23(4):1124–1135, 2002. Study on orthogonal matrices.
- [5] AT&T Bell Labs. Onetone2 matrix dataset, 1993.
- [6] Youcef Saad. *SPARSKIT: A Basic Tool Kit for Sparse Matrix Computations*. University of Minnesota, 1994. Toolkit for handling sparse matrix computations.

Appendix

Algorithm 1 Generate Multiple Matrices

```
1: Input: Original matrix  $A$ , dimensions (rows, cols), density,  $N$ , weights
2: Output: Generated matrices and their loss values
3: Compute properties of  $A$ :  $orig\_props \leftarrow compute\_matrix\_properties(A)$ 
4: Initialize  $generated\_mats \leftarrow []$ ,  $all\_props \leftarrow [orig\_props]$ 
5: for  $i \leftarrow 1$  to  $N$  do
6:   Generate new matrix  $expanded\_matrix$ 
7:   Append  $expanded\_matrix$  to  $generated\_mats$ 
8:   Compute and append its properties to  $all\_props$ 
9: end for
10: Compute scaling dictionary  $scaling\_dict$  from  $all\_props$ 
11: for matrix in  $generated\_mats$  do
12:   Compute properties and loss
13:   Save matrix to file
14: end for
15: Find the matrix with the minimum loss
16: Return:  $generated\_mats$ , loss values
```

Algorithm 2 Optimize Multiple Matrices

```
1: Input: Original matrix  $A$ , initial matrices  $M$ , weights, scaling dictionary,  $max\_iters$ ,  $step$ 
2: Output: Optimized matrices, best loss
3:  $orig\_props \leftarrow compute\_matrix\_properties(A)$ 
4:  $best\_mats \leftarrow copy(M)$ ,  $best\_score \leftarrow total\_loss(best\_mats)$ 
5: for  $iteration \leftarrow 1$  to  $max\_iters$  do
6:   Randomly select  $M_k \in best\_mats$ 
7:    $perturbed\_matrix \leftarrow random\_perturb\_matrix(M_k, step)$ 
8:   Replace  $M_k$  with  $perturbed\_matrix$ 
9:    $new\_score \leftarrow total\_loss(best\_mats)$ 
10:  if  $new\_score < best\_score$  then
11:     $best\_score \leftarrow new\_score$ 
12:  else
13:    Revert  $M_k$ 
14:  end if
15:   $step \leftarrow step \times 0.999$ 
16: end for
17: Return:  $best\_mats$ ,  $best\_score$ 
```
